

VirtualSTEMExplorerTK — User Guide & Technical Documentation

VirtualSTEMExplorerTK is a standalone Python/Tkinter desktop application designed for real-time, interactive visual exploration and measurement of 4D-STEM (4D Scanning Transmission Electron Microscopy) datasets.

It integrates high-resolution virtual image reconstruction, flexible detector masking (annular, center of mass, fluctuation), real/reciprocal space metric rulers, calibration database synchronization, and publication-ready figure exporting.

1. Installation & Prerequisites

System Requirements & Dependencies

VirtualSTEMExplorerTK requires Python 3.9+ and the following scientific packages:

nbed: Container library exposing **pyNBED** (HDF5 DECTRIS loader) and **MicroscopeCalibrationManager**.

I recommend installing miniconda and creating a separate Python environment for nbed. You can download and install Miniconda by following the official [Anaconda Installation Guide](#).

Once you have installed miniconda, open a miniconda or anaconda terminal and create and activate the nbed environment:

```
conda create -name nbed
conda activate nbed
```

You may have to install python:

```
conda install python
```

Install nbed from GitHub (Note: On windows you may need to install the git utilities: Download the official installer from the [Git for Windows Website](#)):

```
python -m pip install git+https://github.com/LotharHouben/nbed.git@main
```

Download the Microscope calibration database file **MicroscopeCalibrationDB.json** from <https://github.com/LotharHouben/nbed/tools> and save it in a path of your choice.

You can then start VirtualSTEMExplorerTK from its installation path:

```
python -m VirtualSTEMExplorerTK
```

2. Configuration & Preferences Management

Configurations are persisted across sessions in an OS-standard location:

- **Linux/macOS:** `~/.config/VirtualSTEMExplorerTK/VirtualSTEMExplorerTK_config.json`
- **Windows:** `%APPDATA%\VirtualSTEMExplorerTK\VirtualSTEMExplorerTK_config.json`

Accessing Preferences

Click  **Settings...** in the top toolbar (or select **File → Settings...** from the main menu) to open the Preferences dialog:

1. **Microscope Calibration Database:** Browse and set the path to your `MicroscopeCalibrationDB.json`

(see section 9 if you need to generate a database file)
2. **Colormap & Mask Defaults:** Define default colormaps for both panels and set default spatial ROI (dx,dy) and detector mask radii (rin ,rout).
3. **Default Import Binning:** Set standard scan-space (Y,X) and detector-space (Y,X) binning factors for fast initial dataset loading.

3. Loading Datasets

You can launch the GUI programmatically from Python or load files interactively via the UI:

Programmatic Launch

Python

```
from VirtualSTEMExplorerTK import VirtualSTEMExplorerTK

# Launch empty GUI or load an initial file
VirtualSTEMExplorerTK(initial_filepath="/path/to/
AS130_0013_master.h5")
```

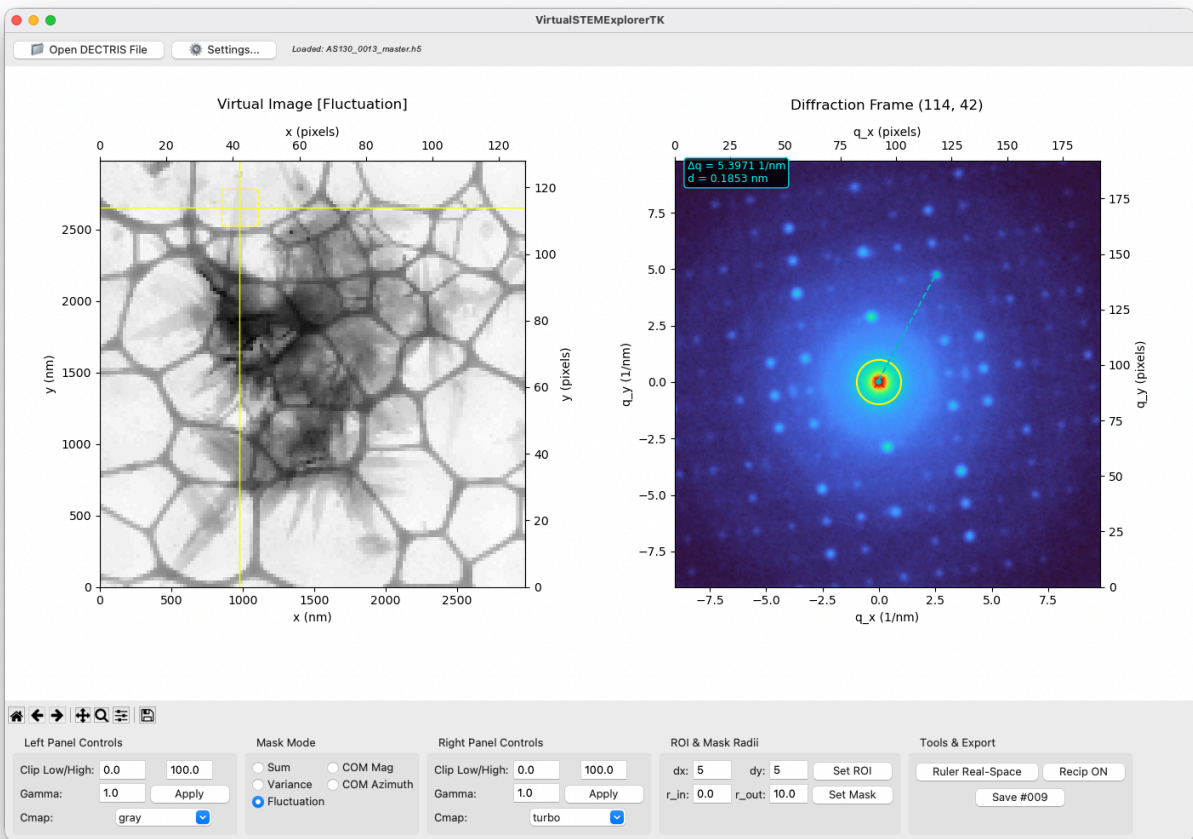
Interactive File Import

1. Click  **Open DECTRIS File** in the top toolbar or menu bar.
2. Select your master HDF5 file (`.h5` / `.hdf5`).
3. Set the dataset binning options in the modal dialog:

- **Real Space Binning (`bin_scan Y, X`):** Reduces real-space scan dimensions for higher processing speeds (e.g., `2, 2`).
 - **Reciprocal Space Binning (`bin_det Y, X`):** Bins detector pixels per frame (e.g., `1, 1`).
4. Click **Load Data**. The tool parses microscope metadata, pulls scale parameters from `MicroscopeCalibrationDB.json`, and computes default reconstructions.

4. Main Explorer Interface Overview

The interface consists of two synchronized viewports and a unified control dock:



5. Viewport Controls & Integration Modes

Left Viewport (Virtual STEM Reconstruction)

- **Probe Selection:** Click anywhere on the left panel image to move the probe position. The right panel instantly updates to display the diffraction pattern averaged over the spatial region ($x \pm dx, y \pm dy$).
- **Contrast Controls:** Adjust **Clip Low/High** percentiles or **Gamma** curve, then click **Apply**.
- **Colormaps:** Select custom colormaps (e.g., `gray`, `inferno`, `viridis`, `turbo`, `hsv`).

Virtual Mask Integration Modes

Select integration modes from the **Mask Mode** control box:

Mode	Typical Use Case
Sum	Standard Bright-Field (BF), Annular Dark-Field (ADF), or HAADF imaging

Variance	Medium-Range Order (MRO) in amorphous or disordered materials
Fluctuation	Normalized Fluctuation Electron Microscopy (FEM)
COM Mag	Differential Phase Contrast (DPC) electric field mapping
COM Azimuth	Magnetic domain orientation mapping (auto-sets <code>hsv</code> colormap)

Right Viewport (Reciprocal Detector Frame)

- **Center Alignment:** Click anywhere on the right panel diffraction image to re-center the virtual detector annular mask (`cqx`, `cqy`). The left virtual image updates immediately.
- **Annular Mask Sizing:** Adjust **rin** (inner radius) and **rou** (outer radius) in pixel units in the bottom dock and click **Set Mask**.

6. Interactive Measurement Rulers

VirtualSTEMExplorerTK includes real-time calibrated distance and lattice spacing rulers:

Real-Space Ruler (Ruler Real)

1. Click **Ruler Real** (turns magenta when active).
2. Click **Point A** on the left virtual image, then move your mouse to **Point B** and click again.
3. Displays vector magnitude Δr in physical units (`nm`, $\text{\AA}$, μm) and equivalent pixel counts directly on screen.

Reciprocal-Space Ruler (Ruler Recip)

1. Click **Ruler Recip** (turns cyan when active).
2. Click **Point A** (e.g., central transmitted beam) and **Point B** (e.g., a diffraction spot) on the right panel.
3. Displays scattering vector magnitude Δq (in $1/\text{nm}$) and automatically calculates the corresponding interplanar lattice d-spacing:

$$d = \Delta q^{-1}$$

7. Figure Exporting

Click **Save #XXX** in the bottom-right control panel to export publication-ready vector/raster graphics:

- **Auto-Incrementing Filenames:** Automatically scans the target directory to create incremented filenames (`..._export_001.pdf`, `..._export_002.pdf`).
- **High-Resolution Vector Output:** Preserves double axes, calibrated physical labels, and annotations without cropping UI control widgets.

8. Python API State Synchronization (**target_dict**)

All user actions inside the GUI continuously update the dictionary passed to **target_dict**:

Python

```
state_dict = {}
VirtualSTEMExplorerTK(myset, target_dict=state_dict)

# Access interactive parameters programmatically after GUI
operations:
current_x = state_dict['x']
current_y = state_dict['y']
measured_d_spacing = state_dict['ruler_recip_d']
active_mode = state_dict['mode']
```

Appendix

9. Microscope Calibration Database (**MicroscopeCalibrationDB.json**) Specification

Version: 2.0

File Format: JSON

Purpose: Stores microscope calibration tables, empirical field-of-view (FOV) values, and regression parameters for 4D-STEM real-space and reciprocal-space sampling calculations.

9.1 Structure Overview

The database uses a hierarchical structure where calibration datasets are indexed by **Microscope Model**, **Detector Name**, **Operating Mode**, and **Accelerating Voltage**.

```
Root
├── version (float)
├── instrumenttables (object)
│   ├── <Instrument_Name> (object)
│   │   ├── <Detector_Name> (array of objects)
│   │   │   ├── mode (string)
│   │   │   ├── submode (string)
│   │   │   ├── subsetting (string, optional)
│   │   │   └── voltage (object)
│   │   │       ├── <Voltage_in_Volts> (object)
│   │   │       │   ├── Magnification (object)
│   │   │       │   └── Camerlength (object)
│   │   │       │       ├── unit (string)
│   │   │       │       └── interpolation (string)
```

└ values (object)

9.2. Field Specifications

9.2.1 Root Level

Field	Type	Required	Description
version	float	Yes	Schema version identifier (e.g., 2.0).
instrumenttables	object	Yes	Container holding all registered electron microscope configurations.

9.2.2 Instrument & Detector Hierarchy

- **<Instrument_Name>** (object): Key matching the microscope string in file metadata (e.g., "FEI Tecnai F20", "Titan Cubed 3589").
- **<Detector_Name>** (array of objects): Key matching the detector identifier string in metadata (e.g., "Dectris ARINA Si", "EMPAD"). Each voltage entry in this array corresponds to a specific optical mode configuration.

Configuration Entry Object

Field	Type	Required	Description
mode	string	Yes	Illumination mode (e.g., "STEM").
submode	string	Yes	Optical submode (e.g., "Microprobe", "Nanoprobe").
subsetting	string	No	Secondary submode or aperture configuration (e.g., "ESI-11x7").
voltage	object	Yes	Map of accelerating voltages in Volts (e.g., "200000", "300000").

9.2.3 Calibration Mapping Object (**Magnification** / **Cameralength**)

Under each accelerating voltage entry, calibrations are categorized into two primary imaging types:

1. **Magnification**: Real-space spatial field-of-view (FOV) calibrations in nm.
2. **Cameralength**: Reciprocal-space field-of-view (FOV) calibrations in rec nm.

Field	Type	Required	Description
unit	string	Yes	Physical unit for the total FOV ("nm" for real-space, "1/nm" for reciprocal-space).
interpolation	string	Yes	Linear regression equation used when no exact nominal match is listed in values.
values	object	Yes	Key-value dictionary of empirical calibrations mapping nominal values to physical FOV values.

9.3 Calibration Lookup & Interpolation Logic

When calculating spatial sampling (pixel pitch), `MicroscopeCalibrationManager` follows a two-tier lookup algorithm:

Step 1: Nominal Settings Lookup

- **Real-Space Magnification:** The nominal magnification value `M` (dimensionless) is extracted from metadata.
- **Reciprocal-Space Camera Length:** The nominal camera length `CL` in millimeters (mm) is extracted from metadata.

Step 2: Value Resolution

1. **Exact Match:** The parser checks the `values` dictionary for a key matching the nominal setting string or float value.
2. **Regression Fallback:** If no exact match exists, the parser evaluates the `interpolation` formula.

9.4 Interpolation Formula Specification

The `interpolation` field contains a string expressing the relationship between the nominal setting `x` (`M` or `CL`) and the physical Field of View (FOV):

$$\text{FOV}(x) = a/x + b$$

Syntax Rule

The formula string must follow the format `"x: <expression>"`, where `x` represents the input variable:

```
text"x: 8.532E7/x + 21.207"
```

Formula Evaluation Examples

- **Real-Space FOV Calculation:**
 - **Formula:** `"x: 184700928./x + 102."`
 - **Input:** Magnification `x=100,000`
 - **Resulting FOV:** 1,949.01 nm
- **Reciprocal-Space FOV Calculation:**
 - **Formula:** `"x: 4137.5/x + 0.3165"`

- **Input:** Camera length $x=520.0$ mm
- **Resulting FOV:** $520.04137.5 + 0.3165 = 8.273$ nm⁻¹

9.5 Complete JSON Schema Example

```
{
  "version": 2.0,
  "instrumenttables": {
    "FEI Tecnai F20": {
      "Dectris ARINA Si": [
        {
          "mode": "STEM",
          "submode": "Microprobe",
          "voltage": {
            "200000": {
              "Cameralength": {
                "unit": "1/nm",
                "interpolation": "x: 4137.5/x + 0.3165",
                "values": {
                  "285": 15.036,
                  "520.": 8.246
                }
              },
              "Magnification": {
                "unit": "nm",
                "interpolation": "x: 8.532E7/x+21.207",
                "values": {
                  "10000": 8533.197,
                  "28500": 2976.743
                }
              }
            }
          }
        }
      ]
    }
  }
}
```